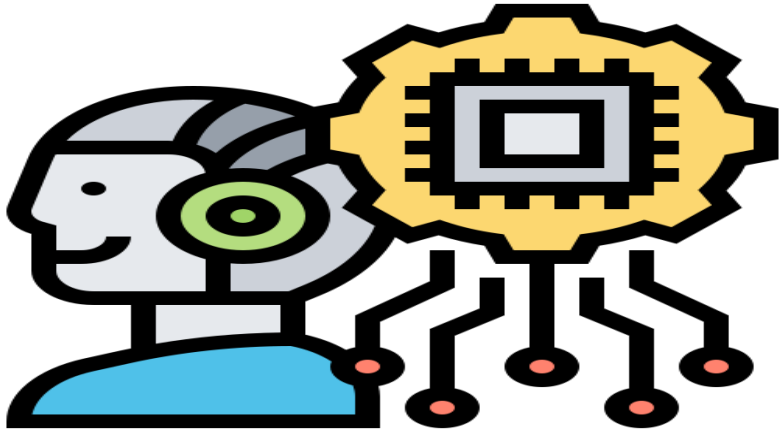
Kakunje Software™

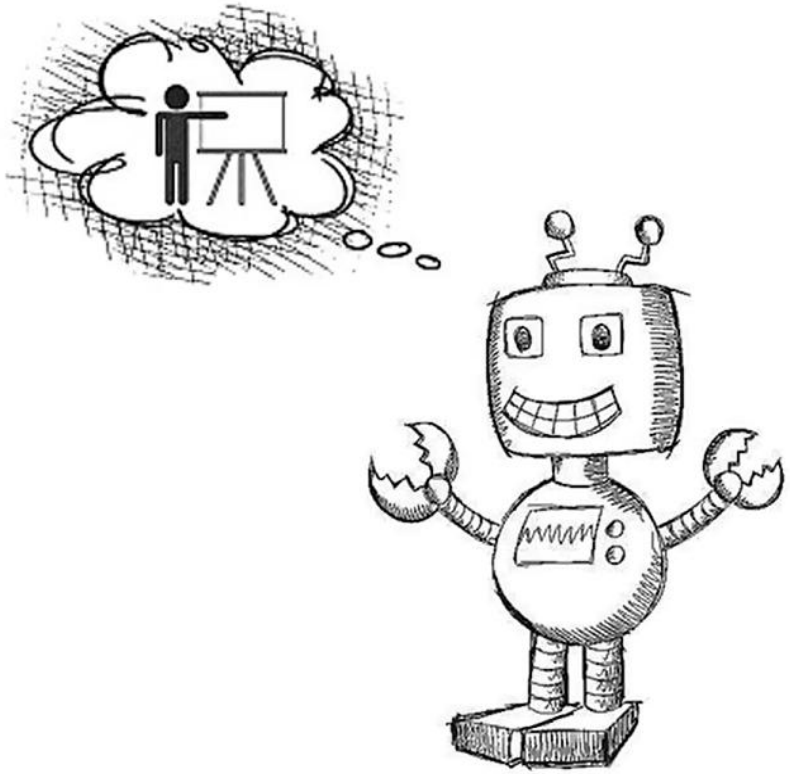
engineering ideas

Introduction to Machine Learning



- Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn from data and make decisions or predictions without being explicitly programmed.
- Machine learning teaches computers to recognize patterns and make decisions automatically using data and algorithms.

Humans learn from past experiences



Machines follow instructions given by humans

Machine Learning Workflow

Data Collection: Gathering data relevant to the problem.

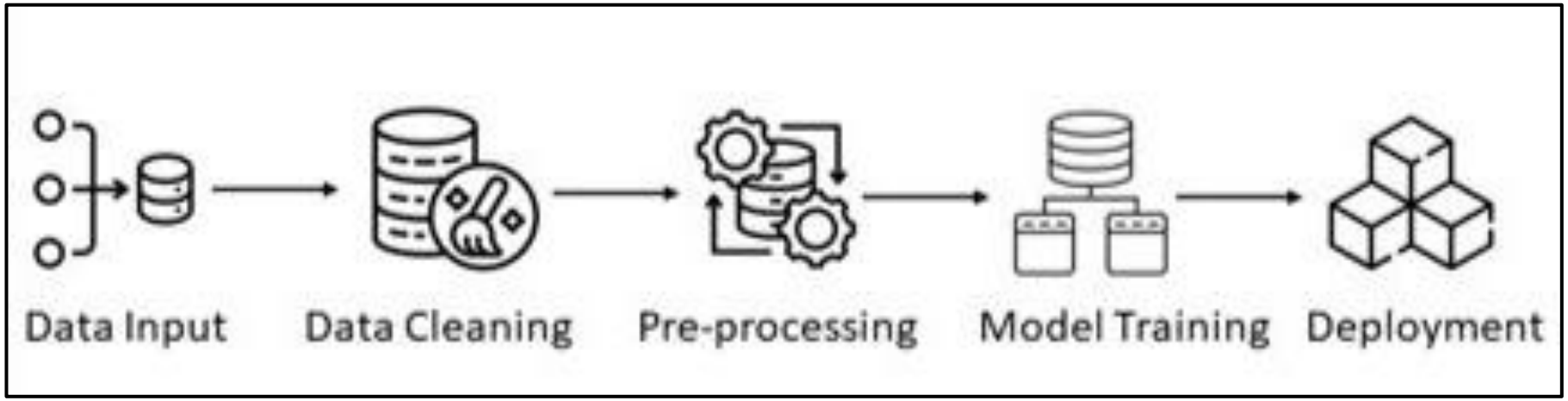
Data Pre-processing: Cleaning and preparing the data for analysis (handling missing values, normalization).

Model Selection: Choosing appropriate algorithms and models.

Model Training: Using data to train the model.

Model Evaluation: Assessing model performance using metrics (accuracy, precision).

Deployment: Integrating the model into a production environment.



Types of Machine learning

Supervised Learning:

- Training a model on labelled data-set.
- Labelled data-sets have both input and output parameters.
- In Supervised learning, the algorithms learn to map points between inputs and correct outputs.
- **Examples:** Classification (spam detection), Regression (calories burnt prediction).

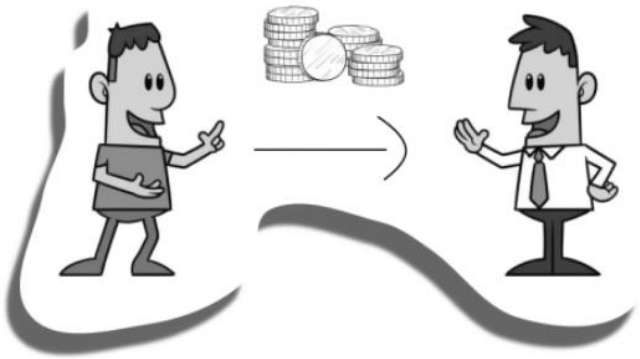
Unsupervised Learning:

- Training a model on unlabelled data to identify patterns or structures.
- The primary goal of unsupervised learning is to group or categories the unsorted dataset according to the similarities, patterns, and differences.
- Machines are instructed to find the hidden patterns from the input dataset.
- **Examples:** Clustering (customer segmentation), Association (market basket analysis)

Reinforcement Learning:

- This works on a feedback based process, in which an AI agent automatically explore its surrounding by hitting and trail, taking action, learning from experiences, and improving its performance.
- In this, there is no labelled data like supervised learning, and agents learn from their experiences only.
- **Example:** Game playing (chess), Robotics.

SUPERVISED LEARNING



3 GRAMS

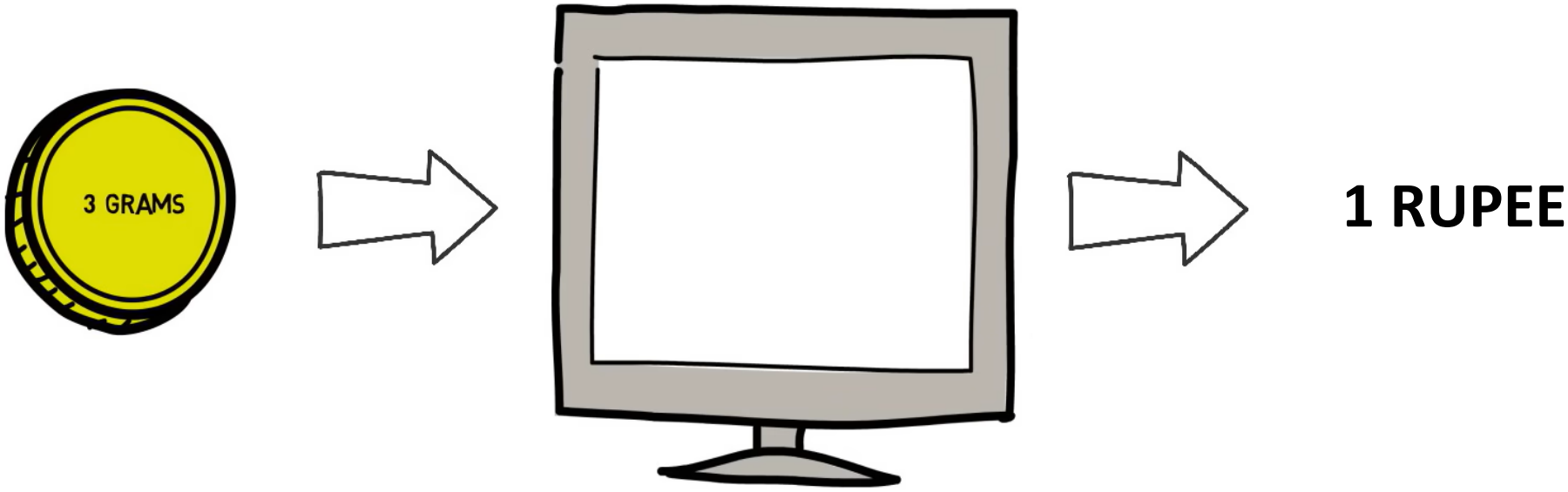
7 GRAMS

4 GRAMS

WEIGHT = FEATURE
CURRENCY = LABEL

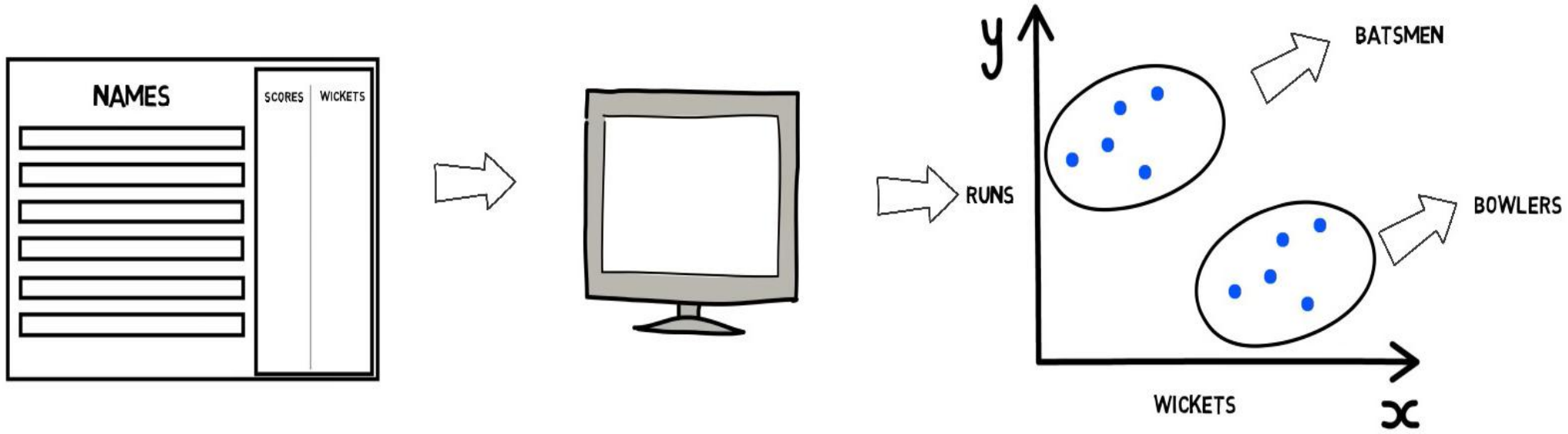
3 GRAMS = 1 RUPEE COIN

SUPERVISED LEARNING



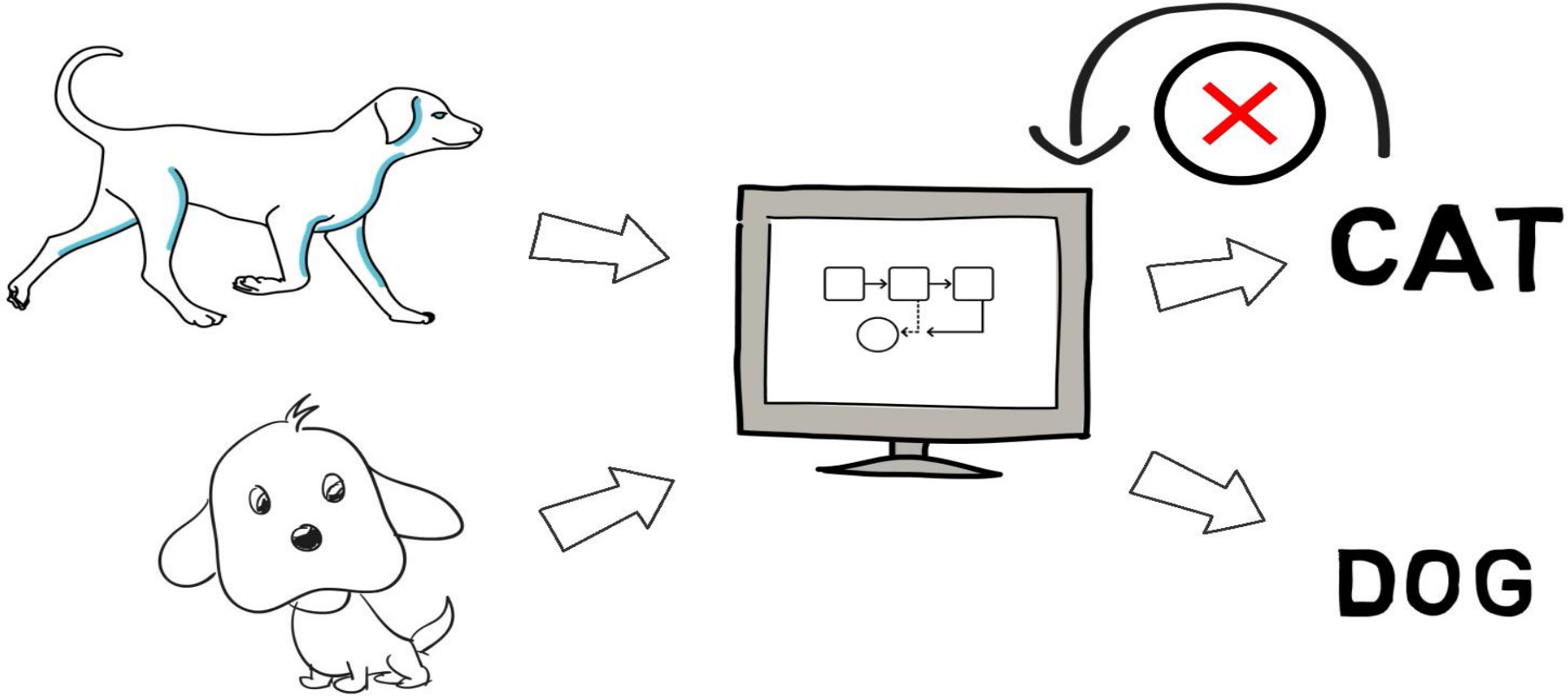
LABELED DATA

UNSUPERVISED LEARNING



NO LABELED DATA

REINFORCEMENT LEARNING



APPLICATIONS OF MACHINE LEARNING

- Healthcare
- Sentiment analysis
- Fraud detection
- E-Commerce
- Image Recognition
- Speech Recognition
- Self Driving Cars
- Stock Market Trading
- Recommender Systems
- Agriculture and Crop Prediction

Machine Learning Module

Scikit-learn

Scikit-learn (sklearn) is a a **powerful machine learning library** for Python. It provides tools for **classification, regression, clustering, dimensionality reduction, and model evaluation**.

INSTALLATION
pip install scikit-learn

Supervised Learning Algorithms:

- K-Nearest Neighbours (KNN)
- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- Naïve Bayes
- Support Vector Machines (SVM)

Unsupervised Learning Algorithms:

Clustering

- K-Means Clustering

Dimensionality Reduction

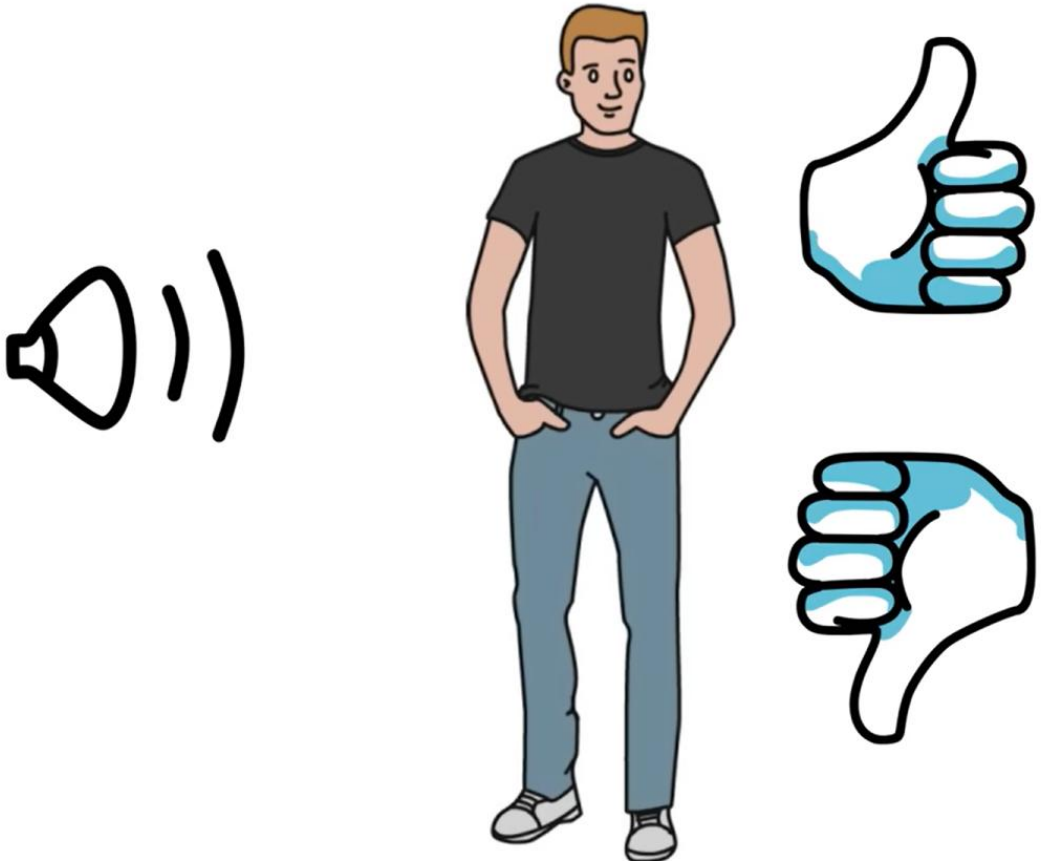
- Principle Component Analysis
- Independent Component Analysis

Association Rule Learning

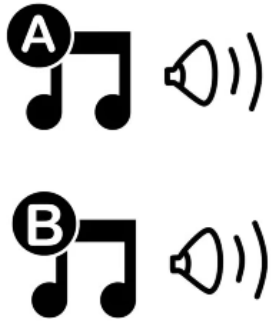
- Apriori Algorithm

K-NEAREST NEIGHBORS (KNN)

SUPPOSE PAUL IS LISTENING TO SONGS…



- **TEMPO**
- **GENRE**
- **INTENSITY**
- **GENDER OF VOICE**

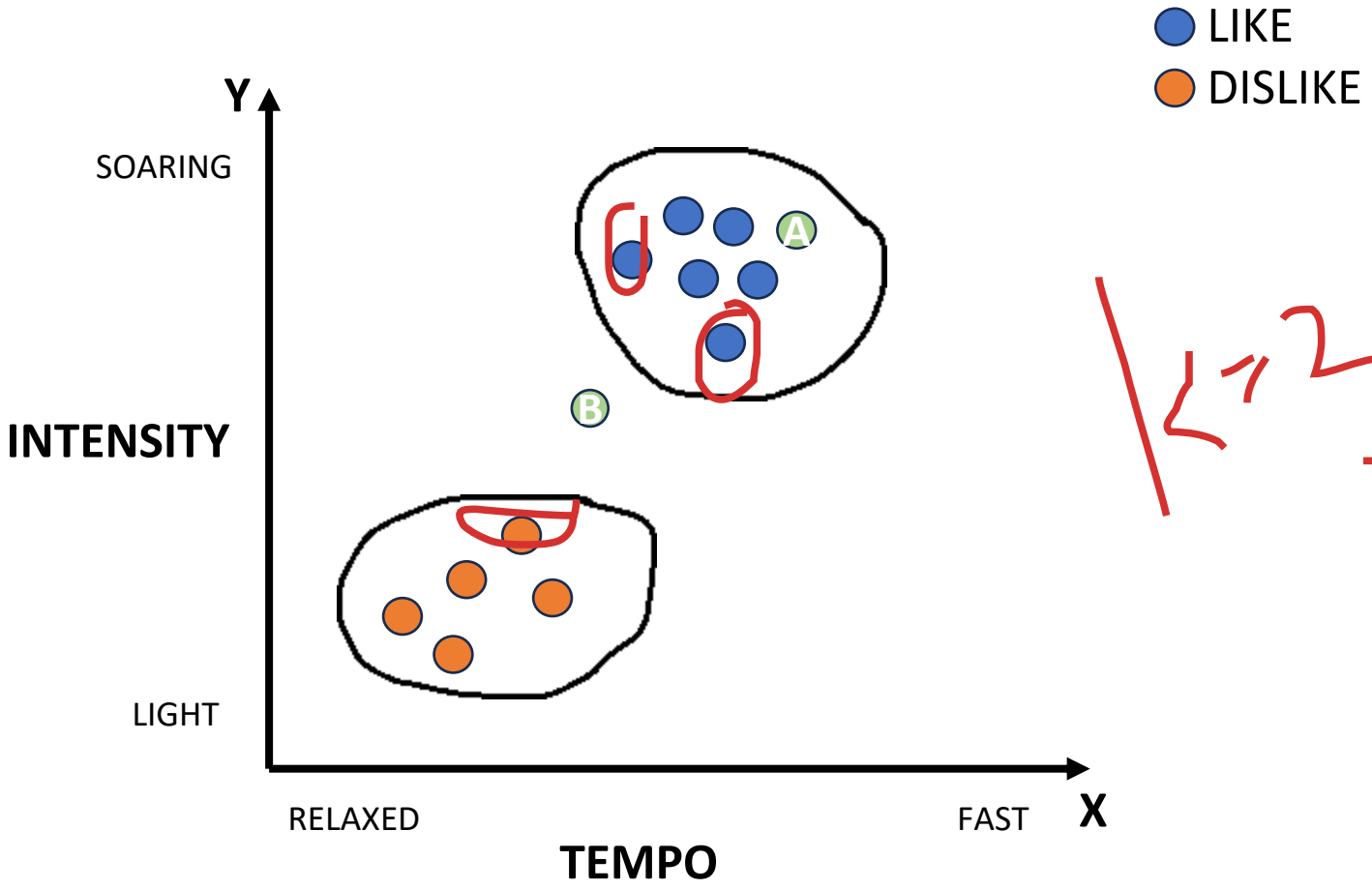


SONG A

- FAST TEMPO
- SOARING INTENSITY

SONG B

- MEDIUM TEMPO
- MEDIUM INTENSITY



- K-Nearest Neighbors (KNN) is a simple way to classify things by looking at what's nearby.
- K-Nearest Neighbors is also called as a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification it performs an action on the dataset.

Working

Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is used. Distance is calculated between data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- When you want to classify a data point into a category (like spam or not spam), the K-NN algorithm looks at the **K closest points** in the dataset. These closest points are called neighbors. The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called **majority voting**.
- In regression, the algorithm still looks for the **K closest points**. But instead of voting for a class in classification, it takes the **average** of the values of those K neighbors. This average is the predicted value for the new point for the algorithm.

Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.neighbors import KNeighborsClassifier

# Load the dataset
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

print(df.describe())  # Summary statistics
print(df["species"].value_counts())  # Count of each species

# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert species names into numerical values
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2
```

```
# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}")

# Define KNN model with k=5
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)

# Predict the test set
y_pred = knn.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")
# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))
# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

MODEL EVALUATION

Classification Metrics

Classification metrics are used to evaluate the performance of a classification model. These metrics help determine how well a model distinguishes between different classes.

- **Accuracy** - Measures the percentage of correctly classified instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Best for: Balanced datasets where classes have nearly equal distribution.

- **Precision (Positive Predictive Value)** - Measures how many of the predicted positive cases are actually positive.

$$Precision = \frac{TP}{TP + FP}$$

Best for: When false positives (FP) are costly (e.g., medical tests, spam detection).

- **Recall** - Measures how many actual positive cases were correctly predicted.

$$Recall = \frac{TP}{TP + FN}$$

Best for: When false negatives (FN) are costly (e.g., fraud detection, disease diagnosis).

- **F1-Score** – Harmonic mean of precision and recall, balancing both.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Best for: When both false positives and false negatives matter.

- **Confusion Matrix**

Actual \ Predicted	**Positive (P)**	**Negative (N)**
Positive (P)	TP	FN
Negative (N)	FP	TN

True Positive (TP): Correctly predicted positive cases.
False Positive (FP): Incorrectly predicted as positive.
False Negative (FN): Incorrectly predicted as negative.
True Negative (TN): Correctly predicted negative cases.

Label Encoder

Label Encoding is a technique used in machine learning to convert categorical data (text labels) into numerical values so that models can process them.

Why we use Label Encoding?

- Many machine learning models like decision trees work with numbers, not text.
- Converts text data into a form that the model understands.
- Reduces memory usage compared to text data.

Example

```
from sklearn.preprocessing import LabelEncoder

# Sample categorical data
colors = ['Red', 'Green', 'Blue', 'Green', 'Red', 'Blue']

# Initialize LabelEncoder
encoder = LabelEncoder()

# Fit and transform the data
encoded_colors = encoder.fit_transform(colors)

print("Original:", colors)
print("Encoded:", encoded_colors)
```

Output

```
Original: ['Red', 'Green', 'Blue', 'Green', 'Red', 'Blue’]
 Encoded: [2 1 0 1 2 0]
```

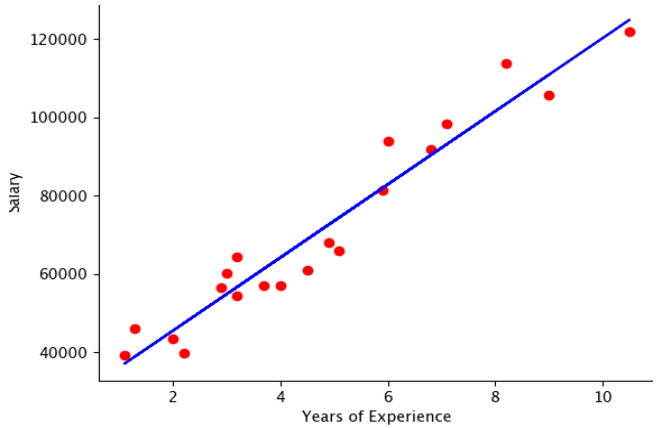
Dependent and Independent Variables

Independent variables: Also called inputs or predictors, do not depend on other features of interest.

Example: Years of Experience

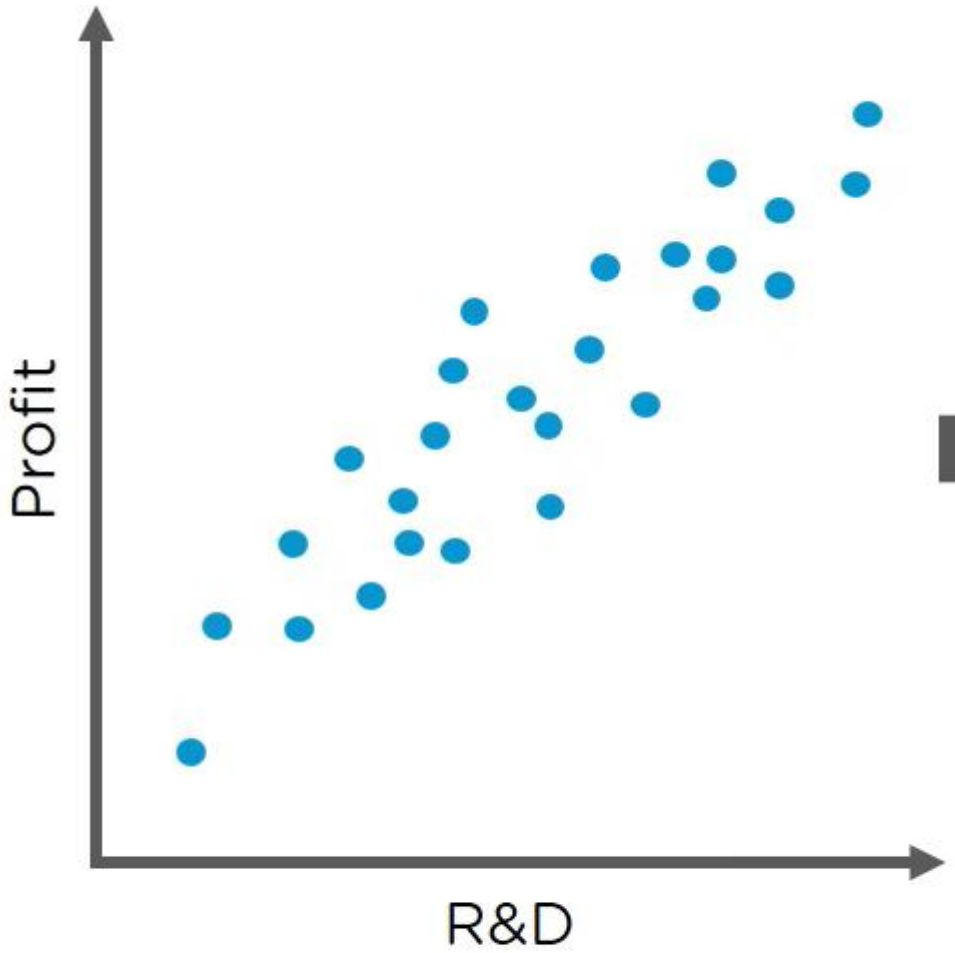
Dependent variables: Also called outputs or responses, depend on the independent variables.

Example: Salary

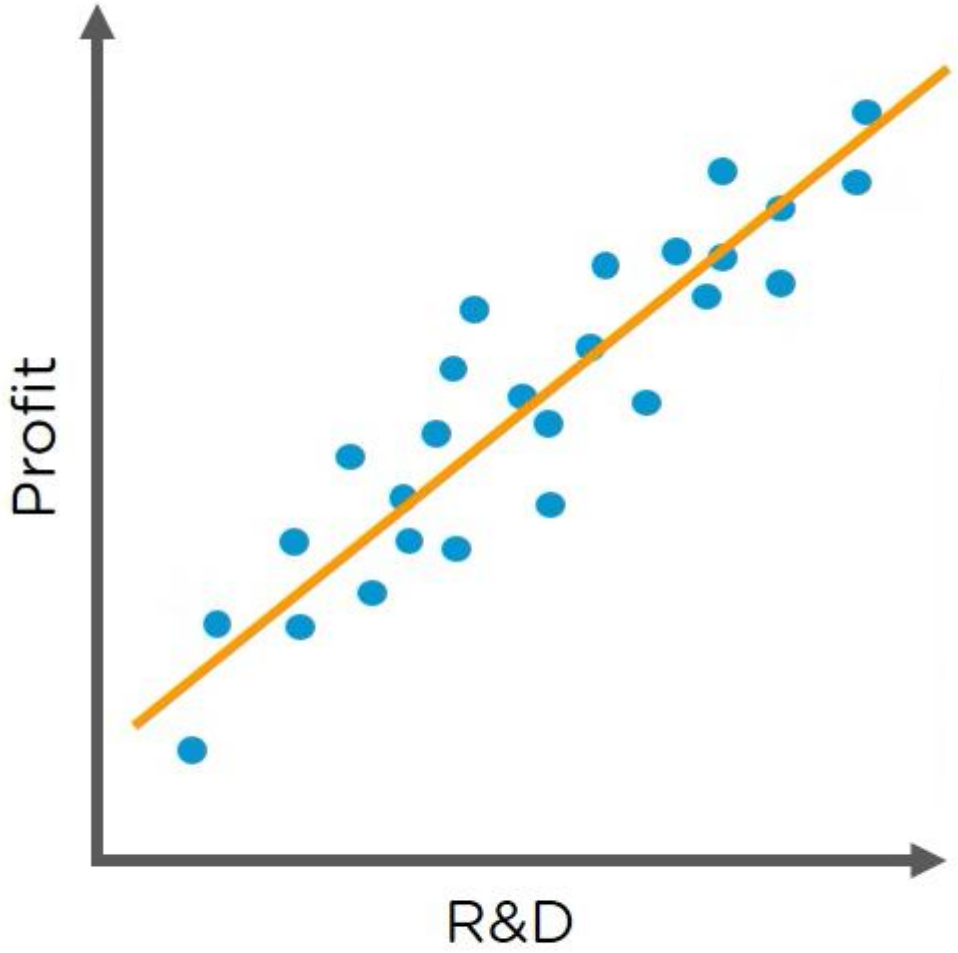


LINEAR REGRESSION

- Linear Regression is one of the most simple Machine Learning algorithm which is used for solving regression problems.
- It is used for predicting the continuous dependent variable with the help of independent variables and relationship between them.
- The goal of the Linear regression is to find the best fit line that can accurately predict the output.
- The relationship should be of linear nature.
- The output for Linear regression should only be the continuous values such as price, age, salary, etc.



Plotting profit based on R&D expenditure



Prediction line to estimate profit

Working

Data with Two variables – You have two variables, such as X (Independent variable) and Y (Dependent variable). The goal of linear regression is to find the best line that fits these points so that you can use it to predict future values of Y.

Fitting a Line – Linear regression looks for the line that best fits the data points. The equation of the line is

$$Y = mX + b$$

* **m** is the slope of the line.
* **b** is the y-intercept.
* **X** is the input, and **Y** is the predicted output

The algorithm works to find the best values for m and b so that the line is as close as possible to all the data points.

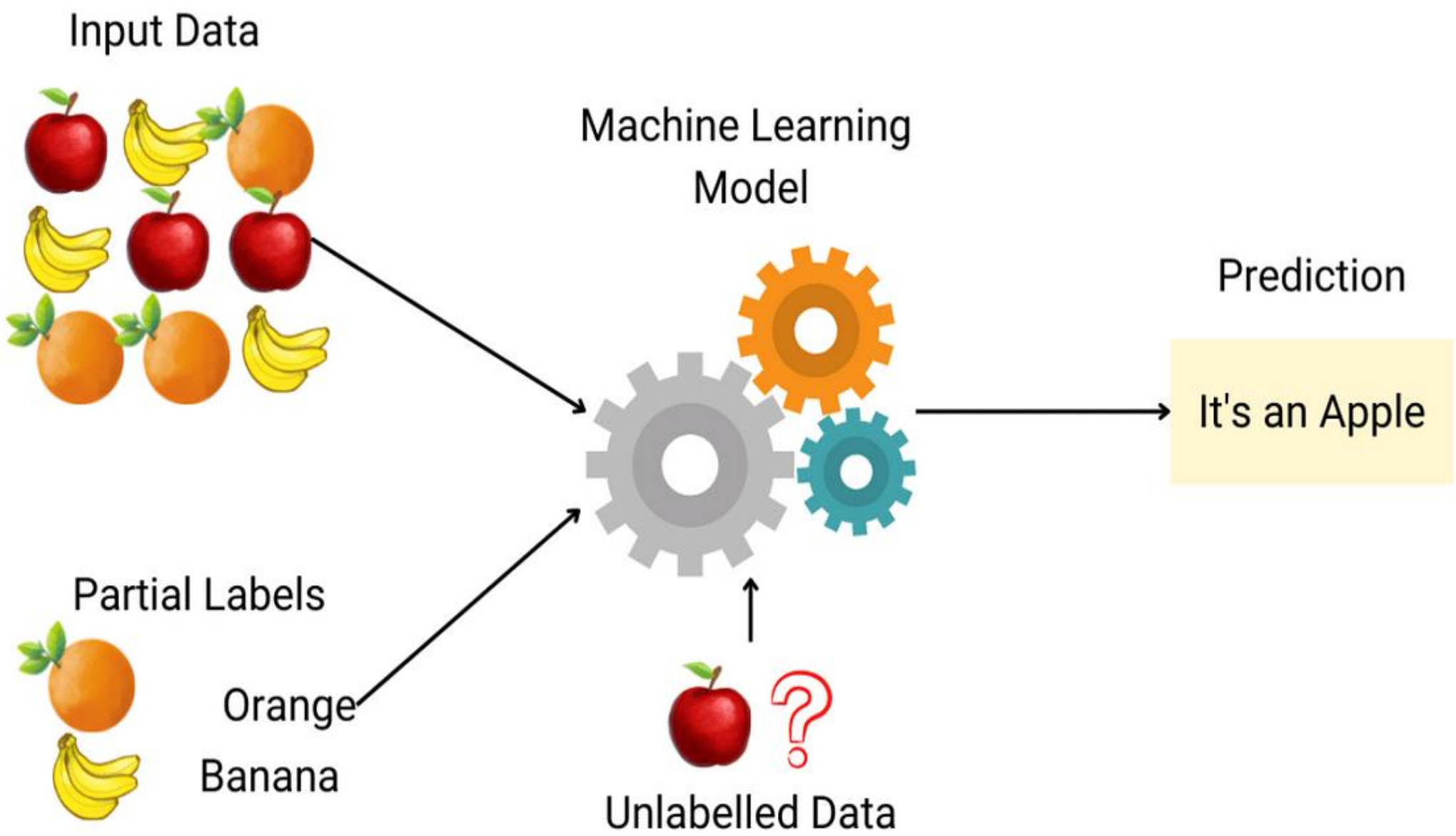
Making predictions – Once you have your best-fit line, you can use it to predict the value of Y for any given X

Example

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

data = pd.read_csv("Housing.csv")
data = data.dropna()

data = data.replace(['yes', 'no', 'furnished', 'semi-furnished', 'unfurnished'], [True, False, 1, 0.5, 0]
print(data.head())

y = data['price']
print(y.head())

x = data.drop(['price'], axis = 1)
print(x.columns)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2)

model = LinearRegression()
model.fit(x_train, y_train)
results = model.predict(x_test)
```

LOGISTIC REGRESSION

- Logistic regression is one of the most popular Machine Learning Algorithm which is used for solving classification problems.
- Logistic regression is used to predict the categorical variable with the help of independent variables.
- The output of Logistic Regression problem can be only between 0 and 1.
- Logistic regression can be used where the probabilities between two classes is required, such as whether it will rain today or not, either 0 or 1, true or false, etc.
- Logistic regression is based on the concept of Maximum Likelihood estimation.


Input Data
Machine Learning Model
Prediction
It's an Apple
Partial Labels
Orange
Banana
Unlabelled Data

Working

Linear Combination of Inputs – Logistic regression starts like Linear regression. It takes the input features and combines them using a weighted sum, similar to how linear regression finds a straight line. The equation looks like this:

$$z = mX + b$$

* **X** is the input data.
* **m** is the weight (slope)
* **b** is the intercept
* **z** is the result of this linear combination

Sigmoid Function – Logistic Regression applies a sigmoid function to the result (z) of the linear combination. The sigmoid function squashes any real number into a value between 0 and 1. The sigmoid function looks like this:

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

The result of this function is a probability between 0 and 1, which tells you how likely it is that an event will occur.

* **If the result is closer to 1:** The event is more likely to happen.
* **If the result is closer to 0:** The event is less likely to happen.

Making Decision – Based on the probability from the sigmoid function, logistic regression then makes a decision:

* If the probability is above a threshold(usually 0.5), it classifies the input as class 1.
* If the probability is below the threshold, it classifies the input as class 0.

Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder


# Load the dataset from CSV
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert categorical species column to numeric
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.linear_model import LogisticRegression

# Initialize Logistic Regression model
model = LogisticRegression(max_iter=200)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)
```

Regression Metrics

Regression metrics help evaluate how well a regression model predicts continuous values. These metrics measure the error between actual and predicted values.

- **Mean Squared Error (MSE)** - Measures the average squared difference between actual and predicted valu

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

Interpretation: Larger errors are penalized more due to squaring.
Best for: When larger error need to be heavily penalized.

- **Mean Absolute Error (MAE)** - Measures the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n}\sum|y_i - \hat{y}_i|$$

Interpretation: Lower MAE means better performance.
Best for: When all errors are equally important.

- **Root Mean Squared Error (RMSE)** – Square root of MSE, bringing error back to original units.

$$RMSE = \sqrt{MSE}$$

Interpretation: Lower RMSE means better performance.
Best for: When large errors should be penalized but still interpretable.

- **R-Squared – Coefficient of Determination** – Measures how well independent variables explain the variance in the dependent variable.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where:

* SS_{res} = Sum of squared residuals
* SS_{tot} = Total sum of squares

Interpretation:

R-square = 1 ⇒ Perfect model
R-square = 0 ⇒ Model explains nothing
R-square < 0 ⇒ Worse than just predicting the mean

Best for: Checking how well the model fits the data.

DECISION TREE ALGORITHM

- Decision tree is a supervised learning technique that can be used for both classification and regression problems, but mostly it is preferred for solving classification problems.

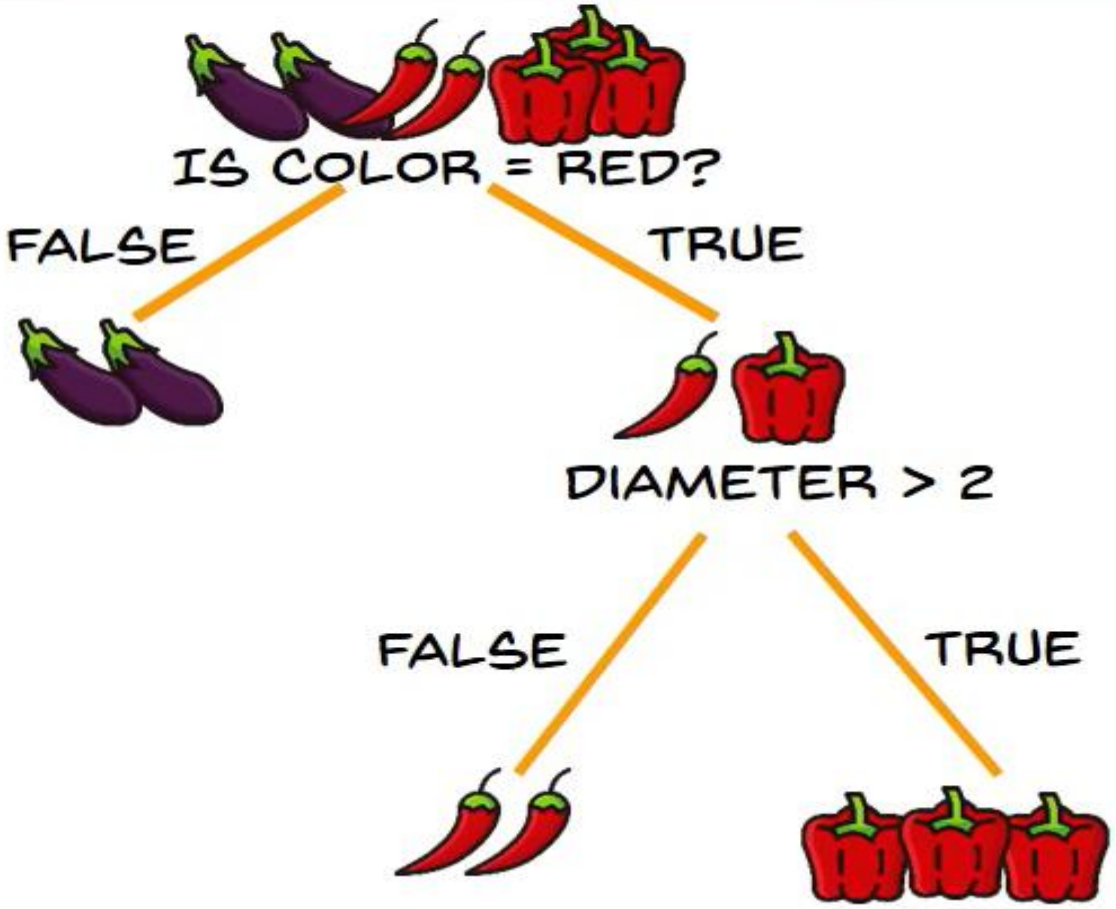
Working

Start with a Question (Root Node) – The tree starts with a big question. This is the root node, the starting point of the tree.

Split based on Answers (Branches and Nodes) – Each question has yes/no (true/false) answers or multiple choices. Based on the answer, the decision moves to the next question (a node). This process continues, splitting the data at each step.

Continue splitting until a Final decision is reached (Leaf Node) – The tree keeps asking questions until it reaches a final decision (a leaf node). This could be a classification.

Which Vegetable?



Terminologies

Root node: Root node is from where the decision tree starts.it represents the entire dataset, which further gets divided into two or more subsets.

Leaf node: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

Decision nodes: Decision nodes are used to make any decision and have multiple branches.

Branch/ sub tree: A tree formed by splitting the tree.

Pruning: Pruning is the process of removing the unwanted branches from the tree.

Parent/ Child node: The root node of the tree is called the parent node, and the other nodes are called the child nodes.

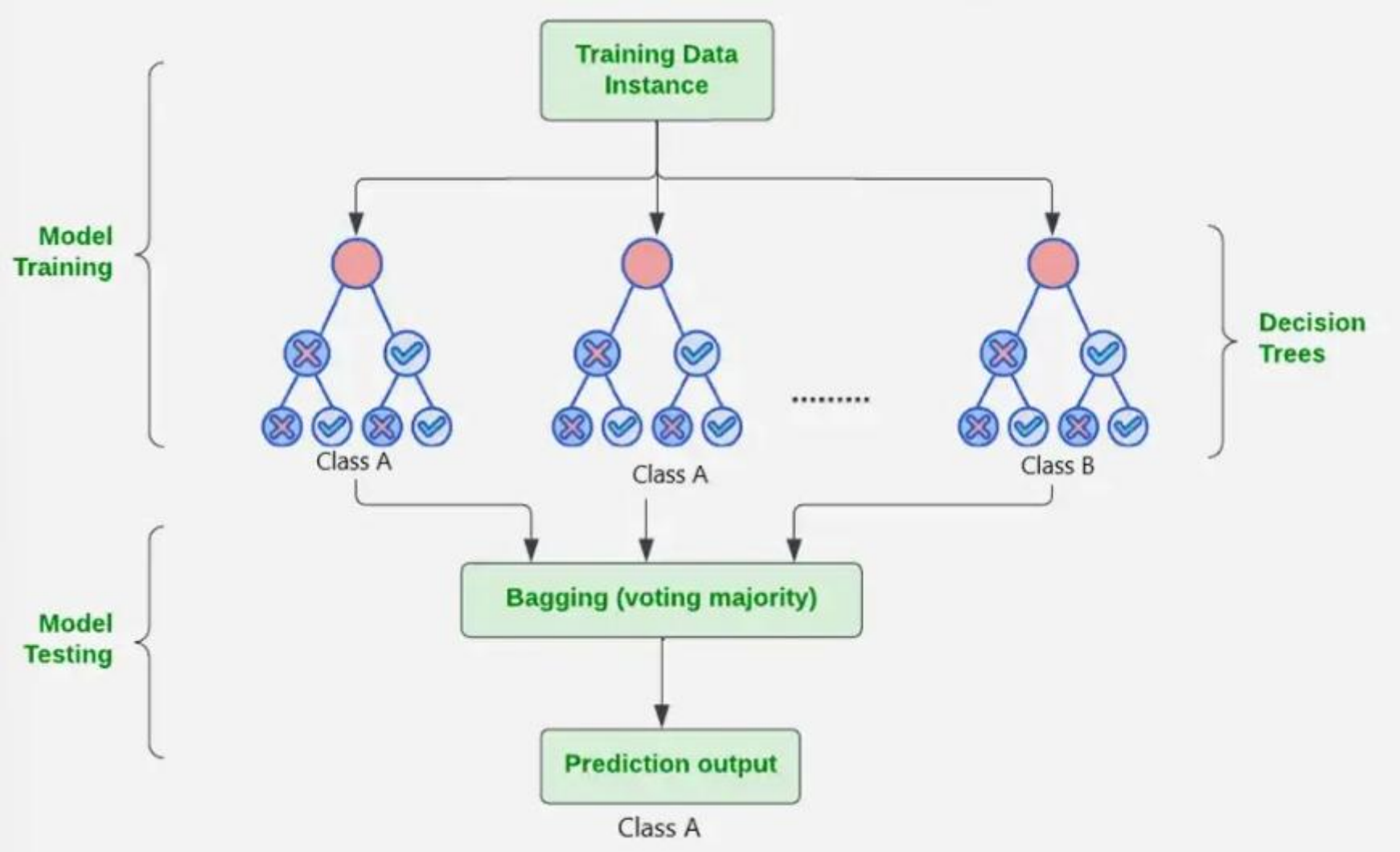
Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load the dataset from CSV
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert categorical species column to numeric
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.tree import DecisionTreeClassifier

# Initialize Decision Tree model
model = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)
```

RANDOM FOREST REGRESSION

- A Random Forest is like a team of decision trees working together to make a better prediction. Instead of using just one decision tree, it uses many trees and takes an average result.
- The Decision of the majority of the trees is chosen by the random forest as the final decision.
- It is used when we want to predict a continuous number.


Training Data Instance
Model Training
Decision Trees
Class A
Class A
.....
Class B
Model Testing
Bagging (voting majority)
Prediction output
Class A

Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load the dataset from CSV
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert categorical species column to numeric
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.ensemble import RandomForestClassifier

# Initialize Random Forest model with 100 trees
model = RandomForestClassifier(n_estimators=100, criterion="gini", random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)
```

NAIVE BAYES ALGORITHM

- Naïve Bayes classifiers are used for classification tasks, works on the principles of conditional probability as given by the Bayes theorem.

Bayes Theorem

$P(A|B) = [P(B|A)P(A)] / P(B)$

where:

$P(A|B)$ = Conditional Probability of A given B

$P(B|A)$ = Conditional Probability of B given A

$P(A)$ = Probability of event A

$P(B)$ = Probability of event B

LET US CONSIDER THE FOLLOWING EXAMPLE OF TOSSING TWO COINS

Here, the sample space is:

{HH, HT, TH, TT}

1. P(Getting two heads) = 1/4
2. P(At least one tail) = 3/4
3. P(Second coin being head given first coin is tail) = 1/2
4. P(Getting two heads given first coin is a head) = 1/2

IN THIS SAMPLE SPACE, LET A BE THE EVENT THAT SECOND COIN IS HEAD AND B BE THE EVENT THAT FIRST COIN IS TAIL

In the sample space:

{HH, HT, TH, TT}

P(Second coin being head given first coin is tail)

= P(A|B)

= [P(B|A) * P(A)] / P(B)

= [P(First coin being tail given second coin is head) * P(Second coin being head)] / P(First coin being tail)

= [(1/2) * (1/2)] / (1/2)

= 1/2 = 0.5

Working

Prepare the data – Collect labelled data and extract features.
Calculate Probabilities using Bayes' Theorem – Compute prior probabilities (how often each class appears). Compute likelihood probabilities (how often features appear in each class)
Make predictions – For a new data point, calculate the probability for each class. Choose the class with the highest probability.
Evaluate the Model – Measure accuracy using test data. Improve performance by tuning features or handling missing data.

Uses of Naïve Bayes:

- Face Recognition
- Weather Prediction
- Medical Diagnosis
- News Classification

Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load the dataset from CSV
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())


# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert categorical species column to numeric
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Initialize Naïve Bayes model
model = GaussianNB()

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

SUPPORT VECTOR MACHINE (SVM)

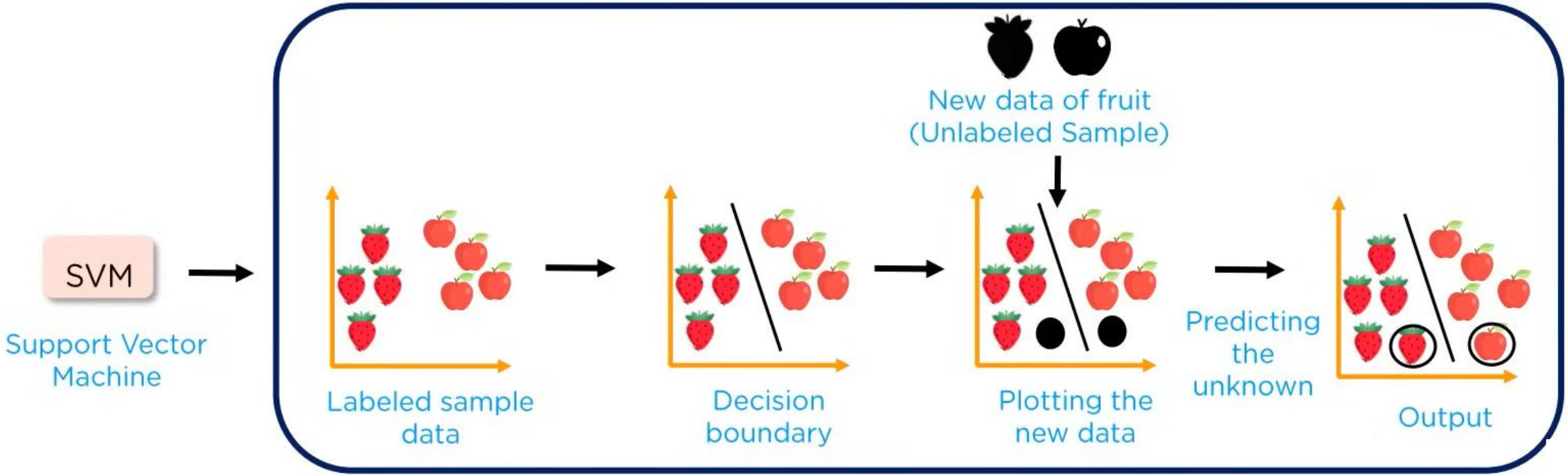
SVM is used for classification and regression tasks, but it is mostly used for classification.

Working

Separating Data with a Line (or Hyperplane) – If you have two types of points on a 2D graph, representing two different categories. SVM tries to draw a straight line (or a hyperplane in higher dimensions) that best separates these two groups.

Finding the Best Line (Maximum Margin) – Among all possible lines that separate the points, SVM chooses the one that maximizes the margin, meaning it keeps the farthest possible distance from the nearest points of both categories. These closest points are called **support vectors**. Sometimes, the data is not neatly separable with a straight line. In such cases, SVM uses a technique called the **Kernel Trick**, which transforms the data into a higher dimension where a separation is possible.

Making Predictions – Once the best boundary (hyperplane) is found, new data points can be classified by checking which side of the boundary they fall on.


SVM
Support Vector Machine
Labeled sample data
Decision boundary
New data of fruit (Unlabeled Sample)
Plotting the new data
Predicting the unknown
Output

Applications

- Face detection
- Text categorization
- Bioinformatics

SVM Components

- Support Vectors: Data points closest to the hyperplane that influence its position.
- Hyperplane: A decision boundary that separates different classes in feature space.
- Margin: The distance between the hyperplane and the nearest data point from either class.

Example

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load the dataset from CSV
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

# Separate features and target variable
X = df.drop(columns=["species"])  # Features (sepal/petal length & width)
y = df["species"]  # Target (flower species)

# Convert categorical species column to numeric
le = LabelEncoder()
y = le.fit_transform(y)  # Converts "setosa", "versicolor", "virginica" to 0, 1, 2

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Initialize SVM model with linear kernel
model = SVC(kernel="linear", C=1.0, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")
# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))
# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Introduction to Text Mining

- Text Mining (also called Text Analytics) is the process of extracting meaningful information from unstructured text data.
- It helps in transforming a raw text into structured data for analysis, insights, and decision making.

Flow of Text Mining

- Text Preprocessing
- Feature Extraction
- Text Analysis
- Text Classification & Clustering
- Visualization
- Interpretation

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF is a numerical statistic used in text mining to measure how important a word is in a document relative to a collection of documents.
It helps in identifying significant words while reducing the weight of commonly used words like “the”, “is”, “and”, etc.

Working
TF-IDF is a combination of two components:

- **Term Frequency (TF):** Measure how often a word appears in a document.

$$TF = \frac{\text{Number of times a word appears in a document}}{\text{Total words in the document}}$$

Higher TF means the word appears frequently in the document.

- **Inverse Document Frequency (IDF):** Measures how important a word is by reducing the weight of common words.

$$IDF = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing the word}}\right)$$

Words that appear in many documents get a **low IDF** (less important).
Rare words get a **high IDF** (more important).

- **TF-IDF Score:**

TF-IDF = TF x IDF

High TF-IDF score: Important and unique word in the document.
Low TF-IDF score: Common and less significant word.

Example

```
from sklearn.feature_extraction.text import TfidfVectorizer
# Sample text documents
documents = [
    "The cat sat on the mat.",
    "The dog sat on the mat.",
    "The cat and dog are friends."
]
# Create TF-IDF Vectorizer
vectorizer = TfidfVectorizer()
# Compute TF-IDF scores
tfidf_matrix = vectorizer.fit_transform(documents)
# Get feature names (words)
feature_names = vectorizer.get_feature_names_out()
# Convert to DataFrame for better readability
import pandas as pd
df = pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names)
# Display the TF-IDF scores
print(df)
```

UNSUPERVISED MACHINE LEARNING ALGORITHMS

CLUSTERING ALGORITHM

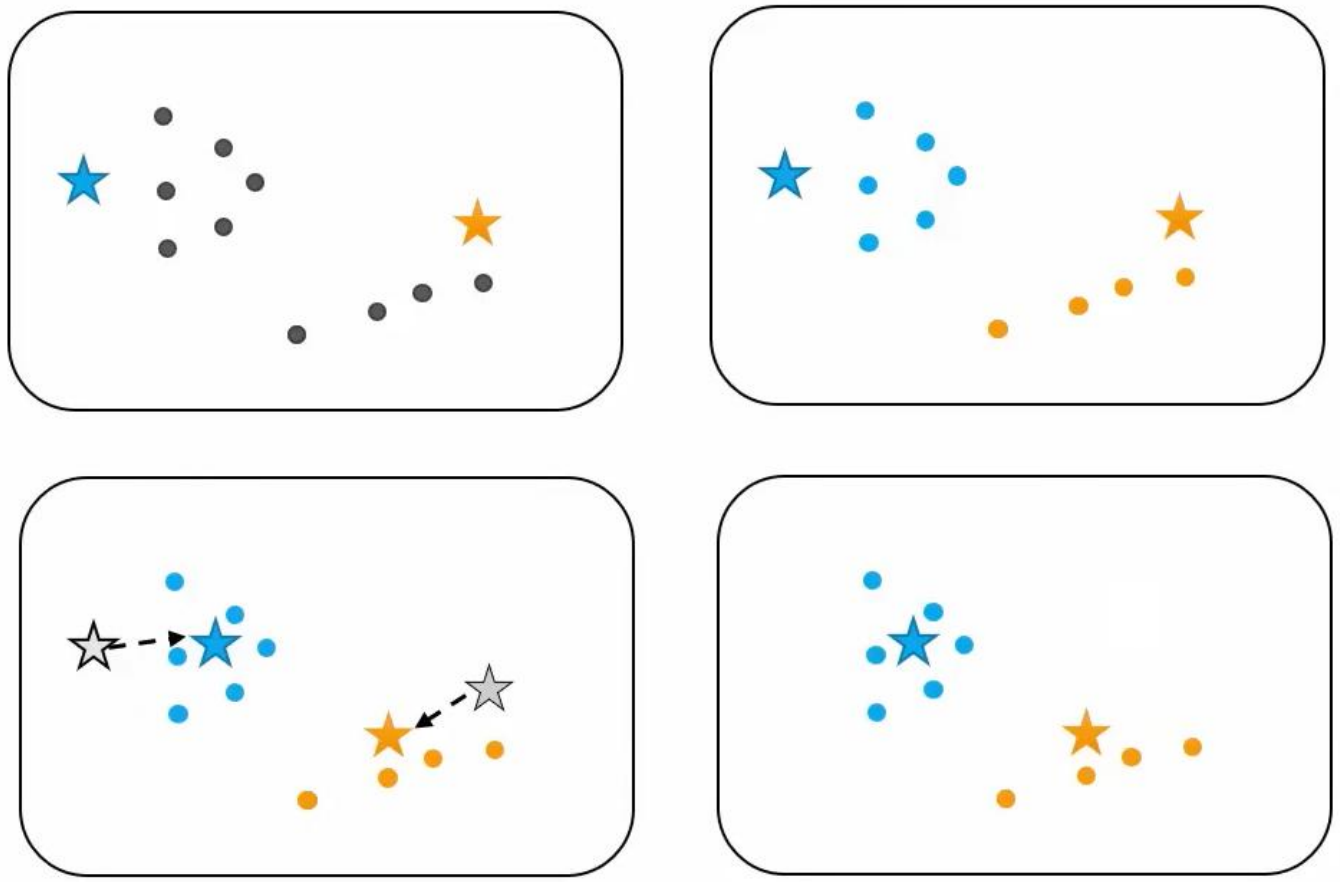
K MEANS CLUSTERING

- K Means clustering is an unsupervised learning algorithm which groups the unlabeled dataset into different cluster.
- The algorithm operates iteratively, minimizing the variance within each cluster.

Working

- **You have a Bunch of Dots** – Suppose you have a lot of data points (dots) on a graph, and you want to divide them into groups based on how close they are to each other.
- **Pick Random Centers (K Centers)** - You first decide how many groups (clusters) you want, let's say 3. The algorithm randomly selects 3 points (centroids) as the starting "centers" of these groups.
- **Assign each dot to the nearest center** - Every data point is assigned to the closest center, forming 3 clusters.
- **Move the Centers** - The center of each cluster is then moved to the average position of all the points in that cluster.
- **Repeat until it Settles** - Steps 3 and 4 are repeated until the centers stop changing much. This means the clusters are now stable.

Pick Random Centers



Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate synthetic dataset with 4 clusters
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)

# Plot the dataset
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], s=50, alpha=0.7)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Generated Cluster Dataset")
plt.show()

# Define K-Means with 4 clusters
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
# Train model
kmeans.fit(X)
# Get cluster labels
clusters = kmeans.labels_
```

```
plt.figure(figsize=(8, 6))

# Plot the clustered data
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap="viridis", s=50, alpha=0.7)

# Plot cluster centers
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c="red", marker="X", s=200, label="Centroids")

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering on Synthetic Data")
plt.legend()
plt.show()
```

DIMENSIONALITY REDUCTION

PRINCIPLE COMPONENT ANALYSIS

Imagine you have a large table with lots of columns (features), but you want to simplify the data while keeping the most important information. PCA helps by reducing the number of columns while still capturing the key patterns in the data.

Working

- **Start with data** - You have a dataset with many features (e.g., height, weight, age, income, etc.).
- **Standardization** - Since features may have different scales (e.g., weight in kg vs. income in dollars), PCA first standardizes them to have the same scale.
- **Find New Axes** - PCA looks for new directions (called principal components) that best capture the variation in the data. These are like new "super features" that combine information from multiple original features.
- **Rank the Components** - It ranks these new features based on how much information (variance) they capture.
- **Select Top Components** - Instead of keeping all original features, you pick the top few principal components that explain most of the data's variation.
- **Transform the Data** - The data is now represented in terms of these fewer components, making it simpler and easier to analyze.

Example

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs
from sklearn.decomposition import PCA

# Generate synthetic dataset with 6 features and 4 clusters
X, y = make_blobs(n_samples=300, centers=4, n_features=6, random_state=42)

# Print dataset shape
print(f"Original dataset shape: {X.shape}")  # (300 samples, 6 features)

# Reduce dimensions from 6 to 2
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Print transformed dataset shape
print(f"Transformed dataset shape: {X_pca.shape}")  # (300 samples, 2 features)
```

```
# Scatter plot of PCA-transformed data
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette="coolwarm", alpha=0.7)
plt.title("PCA on Unsupervised Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()

# Print explained variance ratio
explained_variance = pca.explained_variance_ratio_
print(f"Explained Variance: {explained_variance * 100}")

# Plot explained variance
plt.figure(figsize=(8, 5))
plt.bar(range(1, len(explained_variance) + 1), explained_variance * 100, alpha=0.7, color="blue")
plt.xlabel("Principal Components")
plt.ylabel("Explained Variance (%)")
plt.title("Explained Variance by PCA Components")
plt.show()
```

INDEPENDENT COMPONENT ANALYSIS (ICA)

- Independent Component Analysis is a technique used to separate mixed signals into their independent sources.
- The application of ICA ranges from audio and image processing to biomedical signal analysis. The article discusses about the fundamentals of ICA.

Working

- **Data Collection** – You collect the mixed signals. For example, the sound recordings from multiple microphones.
- **Assumption of Independence** – ICA assumes that the underlying sources (like the songs) are independent of each other.
- **Transformation** – It finds a transformation matrix that separates the mixed signals into their independent components (the original signals).
- **Output** – The algorithm outputs the independent components. These are the sources that caused the mixture (e.g. the songs).

Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import FastICA
from sklearn.datasets import make_blobs

# Step 1: Generate two independent sources (clusters)
n_samples = 2000
X, _ = make_blobs(n_samples=n_samples, centers=2, random_state=42)

# Step 2: Create a random mixing matrix
A = np.array([[1, 1], [0.5, 2]])  # Mixing matrix

# Step 3: Mix the signals (create the mixed data)
X_mixed = np.dot(X, A.T)  # Mix the sources

# Step 4: Apply ICA to recover the original sources
ica = FastICA(n_components=2)
X_ica = ica.fit_transform(X_mixed)  # Recover the sources
```

```
# Step 5: Plot the results
plt.figure(figsize=(8, 6))
# Plot the mixed signals (observed data)
plt.subplot(3, 1, 1)
plt.title('Mixed Signals')
plt.plot(X_mixed[:100])  # Show first 100 samples
plt.xlabel('Samples')
plt.ylabel('Mixed Signals')
# Plot the recovered signals (after ICA)
plt.subplot(3, 1, 2)
plt.title('Recovered Sources by ICA')
plt.plot(X_ica[:100])  # Show first 100 samples
plt.xlabel('Samples')
plt.ylabel('Independent Components')
# Plot the original sources (true components)
plt.subplot(3, 1, 3)
plt.title('Original Sources')
plt.plot(X[:100])  # Show first 100 samples
plt.xlabel('Samples')
plt.ylabel('Original Sources')
plt.tight_layout()
plt.show()
```

ASSOCIATION RULE LEARNING

APRIORI ALGORITHM

- The **Apriori Algorithm** is a popular algorithm used in **data mining** to find **frequent itemsets** in a dataset and to discover **association rules**.
- It is mainly used for **market basket analysis** to find patterns like "customers who buy bread are also likely to buy butter.“

In simple words:

- **Apriori** helps in finding associations between items in large datasets, such as products often bought together.
- It works by looking for patterns that appear frequently in the data and uses them to make **predictions** about what items are likely to be purchased together.

Example

Create a List of Items:

- Think of a store's transactions as a list of items purchased together. For example:
  Transaction 1: [Bread, Butter]
  Transaction 2: [Bread, Milk]
  Transaction 3: [Butter, Milk]
  Transaction 4: [Bread, Butter, Milk]

- **Support** of an itemset is the proportion of transactions that contain that itemset. It is calculated as:

$$\text{Support}(A) = \frac{\text{Number of transactions containing } A}{\text{Total number of transactions}}$$

- **Confidence** of a rule tells us how likely it is that the second item is purchased when the first item is purchased. It is calculated as:

$$\text{Confidence}(A \to B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Step 1: Generate Candidate Itemsets (C1):

C1: {Bread}, {Butter}, {Milk}

Step 2: Calculate Support for C1:

Support({Bread}) = 3/4 = 0.75

Support({Butter}) = 3/4 = 0.75

Support({Milk}) = 3/4 = 0.75

Step 3: Generate Candidate Itemsets (C2):

C2: {Bread, Butter}, {Bread, Milk}, {Butter, Milk}

Step 4: Calculate Support for C2:

Support({Bread, Butter}) = 3/4 = 0.75

Support({Bread, Milk}) = 2/4 = 0.50

Support({Butter, Milk}) = 2/4 = 0.50

Step 5: Generate Rules:

{Bread} → {Butter} with confidence = 0.75 / 0.75 = 1.00 (valid rule)

{Bread} → {Milk} with confidence = 0.50 / 0.75 = 0.67 (valid rule)

{Butter} → {Milk} with confidence = 0.50 / 0.75 = 0.67 (valid rule)

Example

```
 pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# Sample Retail Transaction Dataset (for demonstration purposes)
data = {
    'Milk': [1, 1, 0, 1, 0],
    'Bread': [1, 1, 1, 1, 1],
    'Butter': [1, 0, 1, 1, 0],
    'Beer': [0, 1, 1, 1, 1],
    'Diaper': [1, 0, 1, 0, 1]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Apply the Apriori Algorithm to find frequent itemsets with a minimum support of 0.6
frequent_itemsets = apriori(df, min_support=0.6, use_colnames=True)
```

```
# Print the frequent itemsets
print("Frequent Itemsets:")
print(frequent_itemsets)

# Generate association rules from the frequent itemsets with a minimum lift of 1
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Print the association rules
print("\nAssociation Rules:")
print(rules)
```